Kanea Backend

Architecture, Design Rationale & Defense Guide

A companion document to prepare for the project viva / defense

Component Covered: Backend only (FastAPI + AI engine + database)

Companion File: `Kanea_Project_Report.html` (full project report)

Purpose: Explain every backend design decision and why it was made

Version: 2.0.0

Table of Contents

12. Glossary of Key Terms

1. How to Use This Guide

This document exists for one purpose: to give you, the developer, enough command of **why** the backend was built the way it was — not just what it does — that you can answer any panel question confidently. Section 4 walks through the build as a chronological procedure you can recite. Sections 5–9 are deep dives into each backend file with the reasoning behind key lines of code. Section 10 lists the alternatives you rejected and why. Section 11 is the highest-value section for defense prep: it rehearses the questions a panel is most likely to ask and gives you a tight model answer for each.

> **The one idea to hold onto**
>
> The backend is a **hybrid decision engine**: cheap, deterministic keyword rules decide *what kind of question this is and where the answer should come from*, and the LLM is only responsible for *phrasing* that answer in natural language from data it is explicitly given. The LLM never decides routing and never invents facts — it is constrained by the system prompt to only use the outage records and knowledge-base excerpts handed to it. This is the single design decision that everything else in the backend supports, and it is the answer to most "why" questions.

2. The Backend at a Glance

The backend is five Python modules, each with one job. There is no framework magic hidden anywhere — every request you can trace by eye from the file that receives it to the file that answers it.

```
backend/
 ├── main.py            FastAPI app: REST endpoints, WebSocket, auth, rate limitin
 ├── chatbot.py          The decision engine: intent → route → prompt → Groq LLM c
 ├── knowledge_base.py   RAG: TF-IDF search over static ECG knowledge text
 ├── ecg_scraper.py      Live scraper: pulls real outage notices from ecg.com.gh
 ├── database.py         SQLite access layer: 9 tables, all reads/writes go throug
 ├── ecg_knowledge.txt   Static knowledge corpus (billing, safety, meters, procedu
 └── presentation_text.txt   Extracted text from the Phase-1 slide deck (extra RAG
```

File	Responsibility	Depends on
`main.py`	HTTP/WebSocket surface, request validation, auth, rate limits, background scrape scheduling	database, chatbot, ecg_scraper
`chatbot.py`	Classifies intent, decides the query route, builds the system prompt, calls Groq	knowledge_base, database (session history)
`knowledge_base.py`	Loads two text corpora at import time, scores queries against them with TF-IDF	presentation_text.txt, ecg_knowledge.txt
`ecg_scraper.py`	Fetches and parses live HTML from ecg.com.gh into structured outage records	httpx, BeautifulSoup
`database.py`	Owns the SQLite schema and every query; the only file that touches `chatbot.db`	sqlite3 (standard library)

Request Flow — a single chat message, end to end

1. Browser sends `{"message": "..."}` over the open `/ws/{session_id}` WebSocket.
2. `main.py` checks the 20-messages-per-minute session throttle, then echoes the message back as type `"user"` and persists it via `db.save_message()`.

3. `main.py` pulls the current outage list (`db.get_all_outages()`) and calls `chatbot.get_bot_response()` — this single call is where all "thinking" happens.
4. Inside `chatbot.py`: `classify_intent()` scores the message against 7 keyword sets, `classify_query_type()` turns that into a routing decision (`static` / `dynamic` / `escalation`), `_build_rag_context()` fetches supporting documents if needed, and `_build_system_prompt()` assembles all of it into one instruction block for the LLM.
5. The Groq API is called with that system prompt plus the last 4 conversation turns; the raw completion is scanned for the `[ESCALATE_TO_AGENT]` marker and stripped of it before being returned.
6. `main.py` saves the bot reply (getting back a message ID), logs an escalation row if needed, and sends the reply back over the WebSocket together with intent, quick replies, and RAG source metadata.

3. Design Philosophy — Why Hybrid, Not Pure-NLP or Pure-LLM

Before writing any code, the design had to answer one question: **where does routing intelligence live — in the LLM's own judgement, or in code you control?** Three approaches were possible.

Approach	What it means	Why it was rejected / accepted
Pure LLM (send raw message straight to the model, let it decide everything)	No routing layer — the model reads the user's message and free-writes a reply, possibly with tools	Rejected — the model can hallucinate outage times, fees, or procedures it wasn't told, and there is no reliable, auditable trigger for escalation to a human agent. Unacceptable for a utility company where wrong information (e.g. a wrong restoration time) has real consequences.
Pure rule-based / retrieval-only (classic decision-tree chatbot)	Fixed menu of intents, each mapped to a canned response or template	Rejected — brittle and robotic; cannot handle paraphrased questions, cannot hold a natural multi-turn conversation, and every new phrasing needs a new rule.
Hybrid — keyword classification decides *what* and *where from*; LLM decides *how to phrase it*	Deterministic Python code (fast, free, auditable) does routing; the LLM (flexible, natural) does language generation constrained to supplied facts	Chosen — gets natural, empathetic conversation *and* factual control. Routing is 100% predictable and testable without needing an LLM call at all. The LLM's only freedom is wording, not facts.

Why this matters for a viva

If asked "why didn't you just use ChatGPT/an LLM for everything?", the answer is **controllability and correctness for a safety- and money-adjacent domain**: outage restoration times and billing procedures must come from ECG's actual data, not from whatever the model has memorised about generic utilities. The routing layer is what prevents the model from ever being asked to invent that data in the first place.

4. Step-by-Step: How the Backend Was Designed and Built

This is the procedure to recite when asked "walk me through how you built this." It is written as the logical design sequence — each step exists because the previous step exposed a requirement for it.

1 **Define the customer service scope, then design the database schema first**

Before writing any endpoint, the four customer needs (outage status, fault reporting, billing queries, human escalation) were translated directly into database tables: `outages`, `fault_reports`, `escalations`, and `messages`. Designing the schema first — rather than the API — meant every endpoint added afterward had an obvious, already-decided place to read from and write to.

2 **Stand up the transport layer: FastAPI + WebSocket**

FastAPI was chosen for automatic request validation (Pydantic models catch malformed requests before they reach any logic) and native `async` support. A WebSocket endpoint (`/ws/{session_id}`) was added specifically for chat, instead of plain REST, because chat needs the server to push messages to the client without the client re-polling — a single persistent connection instead of one HTTP round-trip per message.

3 **Build the naive version first: keyword classifier only, canned replies**

The very first working version scored the message against keyword lists and returned a fixed template per intent. This proved the plumbing (WebSocket, DB writes, session IDs) worked before any AI was introduced — isolating transport bugs from AI bugs.

4 **Replace canned replies with an LLM, constrained by a system prompt**

Groq's free-tier API (`llama-3.3-70b-versatile`) was integrated because it is fast and free for a student project, and OpenAI-compatible (so the request/response shape is a widely documented standard). At this point the intent classifier's output stopped being the final answer and became an *instruction to the LLM* instead — the seed of the routing design in Section 3.

5 **Split "static" knowledge from "dynamic" live data — introduce RAG**

Once the LLM was in the loop, a new failure mode appeared: it would answer billing questions correctly (general knowledge) but invent outage restoration times (no way for it to know these). This is the exact problem `classify_query_type()` exists to solve. Two data sources were then built: `knowledge_base.py` (TF-IDF search over static ECG text for billing/safety/FAQ answers) and the live outage table (fed by the scraper in step 6) for anything about current power status. The routing instruction told the LLM explicitly which of the two it was allowed to draw from for a given message.

6 Replace seeded/manual outage data with a live scraper

Seed data in `outages` proved the routing worked, but it was static and would go stale. `ecg_scraper.py` was built to fetch real outage/maintenance notices from ecg.com.gh's homepage, its status index page, and all 8 regional pages, parsing the Joomla CMS link structure into structured records (area, type, cause, status). A separate table, `ecg_outages`, keeps scraped data isolated from the manually seeded `outages` table so a scrape failure never wipes the fallback demo data.

7 Automate the scrape and make it resilient

An `asyncio` background task scrapes once at startup and then every 3600 seconds, using `asyncio.to_thread()` so the synchronous BeautifulSoup/HTTP work never blocks the event loop that serves chat requests. Every scrape result — including failures — is written to `ecg_scrape_log` so there is an audit trail of when the live data was last refreshed.

8 Add the escalation and safety-net logic

A hard-coded `DANGER_KEYWORDS` list (fallen wire, electric shock, fire, ...) was added as a final check that runs independently of the LLM's own judgement — if a customer describes a life-threatening situation, escalation is forced in Python code, not left to the model to decide. This is a deliberate defense-in-depth choice: the LLM is also told to add `[ESCALATE_TO_AGENT]` for danger, but the keyword check cannot be "talked out of it" by clever phrasing the way a model's judgement sometimes can be.

9 Add session continuity and crash recovery

Conversation history is kept in an in-memory dict (`_sessions`) for speed during an active session, but every message is also durably written to SQLite. If the backend process restarts mid-conversation, `get_session_history_for_chat()` reconstructs the last 4 turns from the database the next time that session sends a message — so a server restart does not erase the user's context.

10 Add observability: analytics, feedback, and RAG audit tables

To measure whether the chatbot was actually working (not just running), tables were added for thumbs-up/down per message, 1–5 star session ratings, and a log of which knowledge-base documents were used for each reply. `db.get_analytics()` aggregates these into satisfaction score, escalation rate, and intent distribution — turning "does it work?" into a measurable number instead of an opinion.

11 Harden the backend for anything other than a single trusted user

Once the system worked end-to-end, it was audited as if a stranger had access to it: the admin password was moved server-side behind a token-issuing login endpoint, every mutating REST endpoint got IP-based rate limiting via `slowapi`, the WebSocket got a per-session message cap, and every SQL statement was confirmed to use parameterised placeholders. This step is described in full in Section 9.

5. Deep Dive 1 — The Three-Step Decision Engine (chatbot.py)

Step 1 — Intent classification is a scoring problem, not a lookup

`classify_intent()` does not stop at the first keyword match. It scores *every* intent category by counting how many of its keywords appear in the message, then returns whichever category scored highest. This matters because customer messages often contain words from more than one category (e.g. "my meter is sparking" contains both `billing` and `fault_report` / `safety` vocabulary) — a first-match approach would be order-dependent and fragile; a scoring approach picks the category with the strongest overall evidence.

```
def classify_intent(text: str) -> str:
    # count keyword hits per category, ignore 'general' (it has no keywords),
    # return the category with the most hits, or 'general' if nothing matched
    scores = {intent: 0 for intent in INTENT_KEYWORDS}
    for intent, keywords in INTENT_KEYWORDS.items():
        for kw in keywords:
            if kw in text.lower():
                scores[intent] += 1
    scored = {k: v for k, v in scores.items() if k != "general" and v > 0}
    return max(scored, key=lambda k: scores[k]) if scored else "general"
```

Step 2 — Routing overrides intent when it matters

`classify_query_type()` does not simply trust `classify_intent()`'s output. Two overrides exist on top of it:

- **Escalation always wins.** Even if the highest-scoring intent is `billing`, if any escalation keyword is present the query is routed to escalation — a customer asking for a human should never be redirected into a FAQ answer.
- **A dedicated `_DYNAMIC_PHRASES` set can force live-data routing** even when the intent classifier is ambiguous — e.g. "when will power be back" might not clearly out-score other categories on keyword count alone, but it unambiguously needs live outage data, so it is phrase-matched directly.

Everything that is neither escalation nor dynamic falls through to `static` — this includes billing, safety, complaints, and general queries, all of which are answered from the RAG knowledge base rather than invented.

Step 3 — The system prompt is built differently per route

`_build_system_prompt()` does not send the same instructions for every query. For a `dynamic` route the prompt explicitly forbids inventing outage information and instructs the model to say "no active outage recorded" if the area isn't in the live data. For a `static` route it instructs the model to cite the knowledge base and fall back to the general support number (0302 611 611) rather than speculate. This per-route prompt is the mechanism that actually enforces the "LLM only phrases, never invents" rule from Section 3 — it is not automatic, it has to be told every single time.

> **Why `top_n` differs by route (5 for static, 2 for dynamic)**
>
> For a static query, the knowledge base *is* the answer, so more context (top-5 documents) is given. For a dynamic query, the live outage table is the answer and the knowledge base is only "supplemental background" (e.g. general safety advice) — so only 2 documents are pulled to avoid diluting the prompt with irrelevant FAQ text.

6. Deep Dive 2 — The RAG Knowledge Base (knowledge_base.py)

This module answers a specific requirement: how does the chatbot answer "how do I dispute an overcharge?" accurately without the LLM inventing ECG-specific policy? The answer is retrieval — find the actual text describing that policy and hand it to the LLM instead of asking the LLM to know it.

Why TF-IDF over keyword search and why not embeddings

Option	Trade-off
Plain keyword/substring search	Would treat every matching word equally — common words like "power" or "customer" would dominate scoring over specific, meaningful terms like "prepaid" or "reconnection".
TF-IDF (chosen)	Weighs each matched term by how rare it is across the whole corpus (via IDF), so domain-specific terms score much higher than generic words — with zero external dependencies or API cost, computed with the Python standard library only (`re` , `math` , `collections.Counter`).
Vector embeddings + a vector database	Would give better semantic matching (paraphrase-tolerant) but adds an embedding-model dependency, a vector store, and cost/latency — unjustified for a corpus of only ~30 short documents. TF-IDF's precision is sufficient at this scale and keeps the system dependency-free and instantly explainable in a defense.

How the score is computed

$$IDF(term) = \log((N + 1) / (df + 1)) + 1.0$$

where `N` is the total number of documents and `df` is how many documents contain that term. A document's score for a query is the sum, over each shared term, of `min(query_count, doc_count) × IDF(term)` . The `+1` smoothing in both numerator and denominator prevents division-by-zero and stops any single term from producing an infinite or undefined weight. Documents are then ranked by this score and the top- `N` are returned.

Two corpora, one search function

`DOCS = _load_presentation_sections() + _load_ecg_knowledge_sections()` loads two distinct sources at import time: the Phase-1 presentation slide text (project background) and `ecg_knowledge.txt` (14 sections of real ECG policy — billing, prepaid meters, safety, connections, PURC regulation, contact directory). Both are tokenised and merged into a single searchable list, but each document keeps a `source` tag (`presentation` vs `ecg_website`) so retrieved answers can be traced back to where they came from — this is what powers the `/admin/rag-sources` audit endpoint.

7. Deep Dive 3 — The Live Data Pipeline (ecg_scraper.py)

The scraper exists to solve the staleness problem: seeded outage rows would become fiction the moment a real outage was resolved. Rather than requiring ECG to expose an API (none exists), the scraper reads the same public HTML pages a customer's browser would.

Design decisions and the reasoning behind them

Decision	Reasoning
Scrape the homepage, the status index page, *and* all 8 regional pages separately	ECG's site (a Joomla CMS) surfaces different outage notices on different pages — the homepage shows only the newest; older or region-specific notices only appear on their regional page. Hitting all 9 URLs and deduplicating by title is the only way to get full coverage.
Deduplicate using a `seen` set keyed on lower-cased title	The same notice is often linked from both the homepage and its regional page — without dedup, outages would double-count in the live outage list shown to customers.
Fail soft: `_fetch()` returns `None` on any exception instead of raising	A single region page being down or slow must never crash the whole scrape — the other 8 sources should still succeed. This is a defensive-programming choice appropriate for scraping a third-party site you don't control.
Regex-based cause/date/type extraction instead of a fixed HTML structure parse	ECG's article titles are free-text (e.g. "EMERGENCY MAINTENANCE ON ACCRA EAST FEEDER — 16TH JUNE 2026"), not structured data fields — pattern-matching against known phrasing (broken pole, transformer fault, GRIDCo, feeder names) is the only way to extract structured fields from unstructured titles.
A dedicated `ecg_outages` table, separate from `outages`	Keeps the manually seeded demo/fallback data untouched by scrape cycles; `sync_ecg_outages()` only ever replaces rows in `ecg_outages`, and `get_all_outages()` unions both tables for the chatbot and the public `/outages` endpoint.

8. Deep Dive 4 — The Data Layer (database.py)

Why SQLite and not PostgreSQL/MySQL

SQLite requires zero setup (no server process, no connection string, no credentials) and the entire database is a single file (`chatbot.db`) — appropriate for a single-instance student/demo deployment. It fully supports SQL, transactions, and indexes, so the schema design and query patterns transfer directly to a client-server database if the project were scaled up later; only the connection code in `get_conn()` would need to change.

The context-manager pattern

```
@contextmanager
def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row   # rows behave like dicts: row["area"]
    try:
        yield conn
        conn.commit()                # commit only if no exception was raised
    except Exception:
        conn.rollback()              # undo partial writes on failure
        raise
    finally:
        conn.close()                 # always release the connection
```

Every function in `database.py` uses `with get_conn() as conn:`. This guarantees three things automatically for every single database operation in the codebase: a transaction is committed only on success, any error rolls back cleanly instead of leaving a half-written row, and the connection is always closed — without repeating that logic in 20 different functions.

Why every query uses `?` placeholders

`conn.execute("INSERT INTO fault_reports (...) VALUES (?,?,?,?,?,?,?,?)", (reference, ...))` — parameters are always passed separately from the SQL string, never interpolated with f-strings. This is what makes SQL injection structurally impossible in this codebase: the database driver treats parameter values purely as data, never as executable SQL syntax, regardless of what a user types into a fault report or a chat message.

Schema at a glance

Table	Purpose
`messages`	Every chat message (user + bot), with intent tag and timestamp — the source of truth for session history
`message_feedback`	Per-message helpful/unhelpful rating, upserted (one per message per session)
`chat_ratings`	Session-level 1–5 star rating with optional comment
`fault_reports`	Submitted fault reports with a generated reference code (`FAULT-XXXXXXXX`)
`outages`	Seeded/manual outage records (demo + fallback data)
`ecg_outages`	Live scraped outage records — fully replaced on each scrape cycle
`ecg_scrape_log`	Audit history of every scrape: when, how many outages, which regions, success/failure
`escalations`	Sessions flagged for human agent follow-up
`rag_references`	Which knowledge-base documents were used for each bot reply — the audit trail behind `/admin/rag-sources`

9. Deep Dive 5 — The API Layer (main.py)

REST for state, WebSocket for conversation

Endpoints that are one-shot request/response (submit a fault report, fetch outages, log in) are plain REST — this is the correct fit because each call is independent and stateless. Chat is different: it is a long-lived, bidirectional exchange, so it uses one persistent WebSocket connection per session instead of the client polling a REST endpoint every few seconds. Polling would waste bandwidth and add latency to every message; WebSocket lets the server push a reply the instant it's ready.

Auth design: token issuance, not password-per-request

`POST /admin/login` checks the password once and mints a random 256-bit token (`secrets.token_hex(32)`) stored server-side in an in-memory set. Every subsequent admin call sends that opaque token as a Bearer credential, checked by the `require_admin` dependency. This means the actual admin password is transmitted exactly once, is never stored in the frontend, and a leaked token can be revoked independently via `/admin/logout` without changing the password.

Two independent rate limiters, for two different threats

Layer	Mechanism	Threat it addresses
REST endpoints	`slowapi`, keyed by client IP (`5/min` fault reports, `10/min` ratings, `30/min` feedback)	Scripted abuse/spam of public write endpoints from a single source
WebSocket chat	Custom in-process check (`_is_rate_limited`): max 20 messages per rolling 60-second window, keyed by session ID	A single chat client flooding the LLM API with rapid messages (which would also burn through Groq's free-tier quota)

Why the scrape runs as a background `asyncio` task, not inline

The lifespan handler schedules the scrape as a fire-and-forget task at startup and again every hour, rather than scraping inside a request handler. This keeps outage data warm without making any customer wait for a live scrape to finish before they can chat — the scrape and the chat API run concurrently on the same event loop, and the actual HTML-parsing work is pushed to a thread (`asyncio.to_thread`) so it never blocks other requests being served.

10. Design Trade-offs — Alternatives Considered

Decision point	Alternative(s) considered	Why the current choice won
Intent classification	Trained ML classifier (e.g. scikit-learn, spaCy) / fine-tuned model	A keyword scorer needs no training data, no model file, is instantly explainable to a panel, and is fast enough that classification never adds latency. A trained classifier would need labelled data this project doesn't have.
Knowledge retrieval	Vector embeddings + FAISS/Pinecone	Overkill for ~30 short documents; TF-IDF gives adequate precision with zero extra services or cost, and the scoring math is trivial to explain and defend line-by-line.
LLM provider	OpenAI GPT, self-hosted open model	Groq's free tier gave a capable 70B model with very low latency and no cost for a student project; the API is OpenAI-compatible so switching providers later only means changing `GROQ_URL` / `GROQ_MODEL`.
Database	PostgreSQL/MySQL	SQLite needs no server process or network config — appropriate for the deployment scale of this project; the SQL and schema are portable if scaling is ever required.
Real-time chat transport	HTTP long-polling / Server-Sent Events	WebSocket gives full-duplex, low-latency communication in one connection, matching how modern chat UIs behave; SSE is one-directional and polling wastes requests.
Escalation decision	Trust the LLM's judgement entirely	A hard-coded danger-keyword check runs independently of the model so a safety escalation can never be "argued out of" by ambiguous phrasing or a model mistake.

11. Anticipated Defense Questions & Model Answers

Q. Why is the chatbot not "just calling ChatGPT"? What's actually yours?

A. The LLM call is one line; everything around it is the project — the 7-category intent classifier, the static/dynamic/escalation routing decision, the TF-IDF retrieval engine that grounds answers in real ECG content, the live scraper that keeps outage data current, the danger-keyword safety net, and the schema/analytics layer that measures whether it's actually working. The LLM only converts already-decided facts into natural sentences.

Q. How do you stop the AI from making up an outage restoration time?

A. The system prompt for a dynamic query includes the literal live outage list and an explicit instruction: answer only from this data, and if the area isn't listed, say no outage is recorded for it. The prompt is regenerated per-message from the current database state, so the model is never asked a question it has to guess the answer to.

Q. What happens if the Groq API is down or the API key is missing?

A. Two separate fallbacks exist: if no API key is configured, the bot returns a clear setup message instead of crashing; if the API call fails at runtime (network error, timeout, rate limit), the `except` block returns a message directing the customer to the emergency line and forces `[ESCALATE_TO_AGENT]` — the system degrades to "call a human" rather than failing silently.

Q. Why keyword scoring instead of a proper NLP/ML model for intent classification?

A. It requires no training data or labelled examples (which this project doesn't have), is deterministic and instantly testable, and adds no latency. Given the domain has a fixed, well-known vocabulary (outage/fault/billing/safety terms), a scored keyword match reaches acceptable accuracy without the overhead of training and maintaining a model.

Q. How does the chatbot know an outage is real and current, not stale demo data?

A. Two outage tables exist: `outages` (seeded fallback/demo data) and `ecg_outages` (live scraped data, fully replaced every scrape). `get_all_outages()` unions both, and each scraped row carries a `scraped_at` timestamp plus an entry in `ecg_scrape_log`, so freshness is auditable, not assumed.

Q. What if the ECG website's HTML structure changes — does the scraper break?

A. The scraper is intentionally defensive: `_fetch()` catches all exceptions and returns `None` rather than crashing the whole scrape cycle, and link-matching relies on URL substrings (`current-system-status`, `regional-status`) plus outage keyword presence, not on rigid CSS selectors — a moderate markup change is less likely to break it than a selector-based scraper. A structural change severe enough to remove those URL patterns or keywords would require an update, which is a known and stated limitation.

Q. Why TF-IDF instead of embeddings/semantic search for the knowledge base?

A. The corpus is small (~30 short sections) and mostly keyword-distinctive domain terms (prepaid, tariff, reconnection, PURC). TF-IDF gives strong precision at this scale using only the Python standard library — no embedding model, no vector database, no added cost or latency. Embeddings would help with paraphrase tolerance but aren't justified by the corpus size here.

Q. How is admin access secured?

A. The password lives only in a server-side environment variable and is checked once at `/admin/login`. A random 256-bit token is issued and required as a Bearer credential on every admin call thereafter; the password itself never appears in the frontend code or is ever sent again. Tokens can be individually revoked via logout.

Q. How do you prevent abuse of the public endpoints (spam fault reports, chat flooding)?

A. Two independent limiters: `slowapi` rate-limits REST writes by IP (e.g. 5 fault-report submissions per minute), and a custom per-session check caps WebSocket chat at 20 messages per rolling 60-second window — protecting both the server and the Groq API quota from a single abusive client.

Q. What happens to a conversation if the backend server restarts mid-session?

A. Conversation history is written to the `messages` table after every turn, not just kept in memory. On the next message from that session, if the in-memory session store doesn't have it (fresh process), `get_session_history_for_chat()` reloads the last 4 turns from the database, so the user doesn't lose conversational context across a restart.

Q. How is SQL injection prevented?

A. Every single query in `database.py` uses parameterised `?` placeholders with values passed as a separate tuple — never string-formatted into the SQL text. This is a structural guarantee, not a per-endpoint check: there is no SQL string interpolation anywhere in the codebase.

Q. How is a safety-critical message (e.g. "there's a fallen wire") guaranteed to escalate?

A. Two layers, deliberately redundant: the system prompt instructs the LLM to add `[ESCALATE_TO_AGENT]` for danger scenarios, and independently, `is_danger_message()` checks the raw user text against a hard-coded danger-keyword list in Python and forces escalation regardless of what the model outputs. If the model fails to add the marker, the keyword check still catches it.

Q. How would this scale beyond a single-instance demo?

A. The main constraints at scale are the in-memory admin-token set and session-history dict (both process-local — would need to move to Redis/shared cache for multiple server instances) and SQLite (would migrate to PostgreSQL, which the parameterised-query pattern already supports with minimal change). The routing/RAG/scraper logic itself is stateless per request and would scale horizontally unchanged.

Q. Why does the routing instruction change the system prompt instead of just filtering what data is sent?

A. Both happen together: the data sent is filtered (only outage data for dynamic, only knowledge-base excerpts for static), *and* the instruction text explicitly tells the model which source is authoritative and forbids inventing beyond it. Filtering alone reduces the chance of a wrong answer; the explicit instruction is what makes the constraint reliable rather than incidental.

12. Glossary of Key Terms

Term	Meaning in this project
Intent classification	Deciding which category of question a message falls into (outage, billing, fault, safety, escalation, complaint, general) using keyword scoring
Query routing	Deciding which data source should answer the message: live outage data (dynamic), the knowledge base (static), or a human agent (escalation)
RAG (Retrieval-Augmented Generation)	Retrieving relevant text documents first, then giving them to an LLM as context, so the model answers from real supplied facts instead of its own memorised (and possibly wrong) knowledge
TF-IDF	Term Frequency–Inverse Document Frequency: a scoring method that weighs matched words by how rare/distinctive they are across the whole document collection
System prompt	The instruction block sent to the LLM before the conversation, defining its role, the data it may use, and rules it must follow
WebSocket	A persistent, full-duplex network connection allowing the server to push messages to the client without the client re-requesting
Escalation	Handing a conversation off to a human agent, triggered either by explicit request, danger keywords, or unresolved repetition
Rate limiting	Capping how many requests a client can make in a time window, to prevent abuse or accidental overload
Parameterised query	An SQL statement where values are passed separately from the query text, preventing SQL injection
Idempotent scrape sync	`sync_ecg_outages()` fully replaces the `ecg_outages` table each run, so re-running it never accumulates duplicates